

Harika Jupaka

2403A51L31

Batch-52

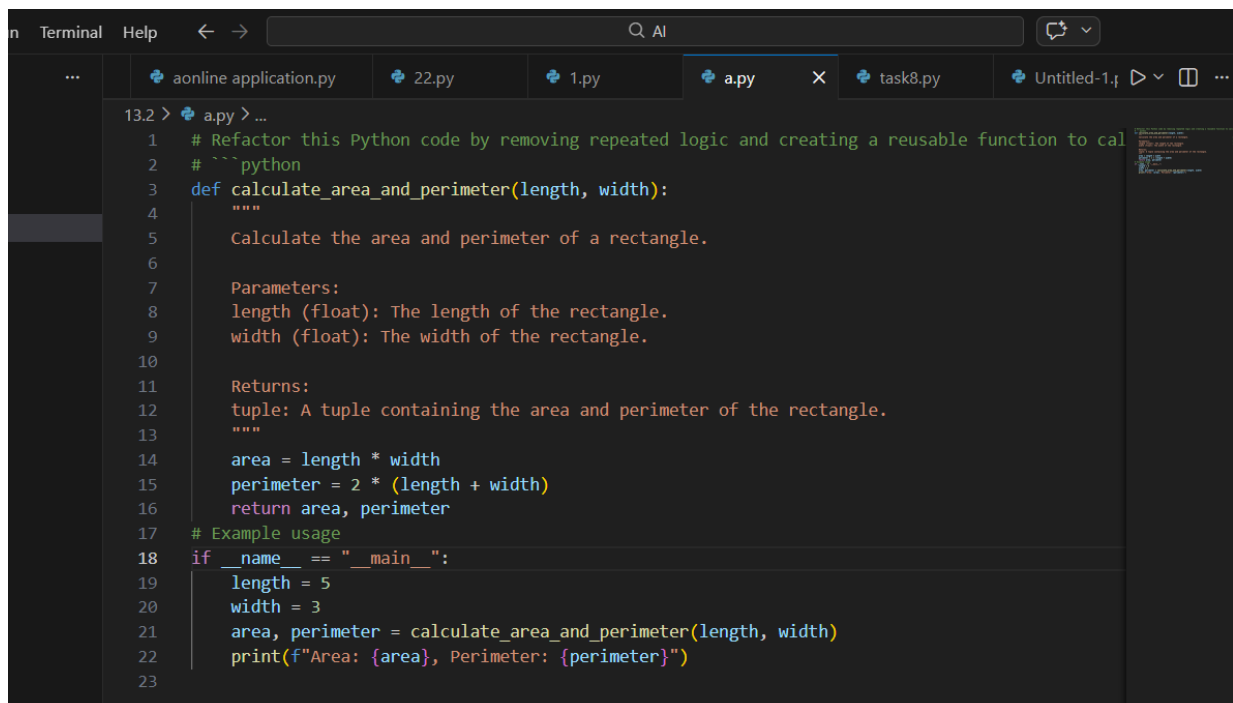
ASSIGNMENT 13.2

Code Refactoring – Improving Legacy Code with AI Suggestions

Task Description #1 (Refactoring – Eliminating Repetitive Logic):

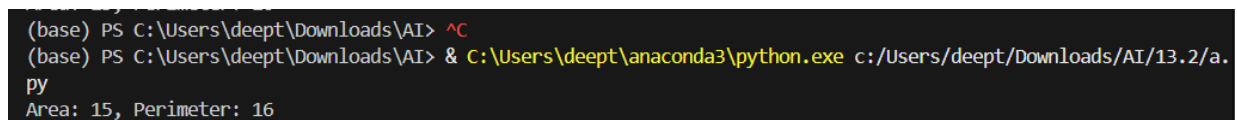
PROMPT: Refactor this Python code by removing repeated logic and creating a reusable function to calculate area and perimeter. Add a docstring and keep the same output.

CODE:

A screenshot of a code editor window with a dark theme. The editor shows a Python file named 'a.py' with the following code:

```
13.2 > a.py > ...
1 # Refactor this Python code by removing repeated logic and creating a reusable function to cal
2 # ``python
3 def calculate_area_and_perimeter(length, width):
4     """
5     Calculate the area and perimeter of a rectangle.
6
7     Parameters:
8     length (float): The length of the rectangle.
9     width (float): The width of the rectangle.
10
11     Returns:
12     tuple: A tuple containing the area and perimeter of the rectangle.
13     """
14     area = length * width
15     perimeter = 2 * (length + width)
16     return area, perimeter
17 # Example usage
18 if __name__ == "__main__":
19     length = 5
20     width = 3
21     area, perimeter = calculate_area_and_perimeter(length, width)
22     print(f"Area: {area}, Perimeter: {perimeter}")
23
```

OUTPUT:

A screenshot of a terminal window with a dark background. It shows the command prompt and the execution of a Python script:

```
(base) PS C:\Users\deept\Downloads\AI> ^C
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.py
Area: 15, Perimeter: 16
```

Explanation :

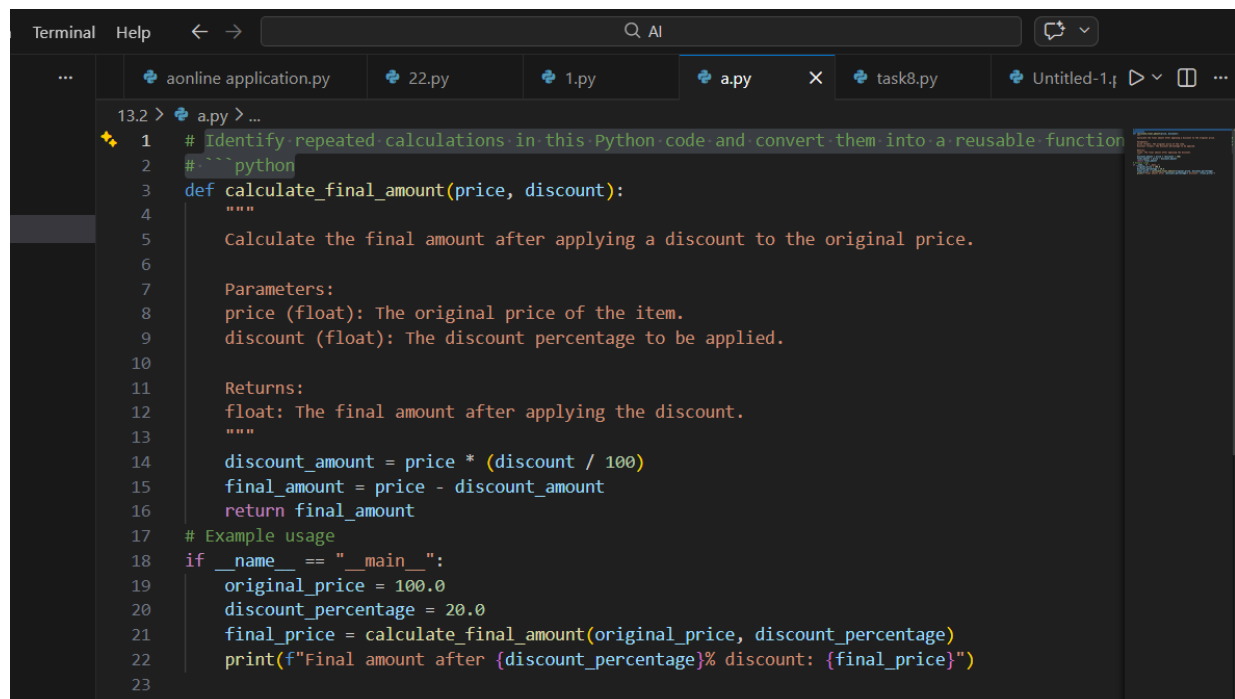
- Identifies repeated area and perimeter calculations.
- Encapsulates repeated logic inside a reusable function.
- Improves maintainability and reduces code duplication.
- Adds docstrings for better documentation.
- Ensures the output remains identical to the original program.

Task Description #2 (Refactoring – Modularizing Inline

Calculations):

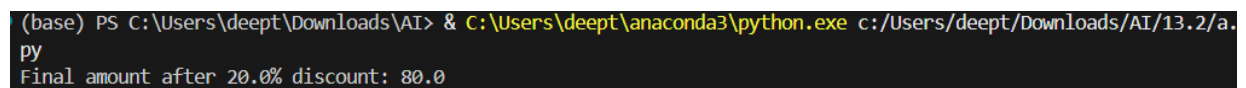
PROMPT: Identify repeated calculations in this Python code and convert them into a reusable function to calculate the final amount after discount. Add a docstring.

CODE:

A screenshot of a code editor window with a dark theme. The editor shows a Python file named 'a.py' with the following code:

```
13.2 > a.py > ...
1 # Identify repeated calculations in this Python code and convert them into a reusable function
2 # ``python
3 def calculate_final_amount(price, discount):
4     """
5     Calculate the final amount after applying a discount to the original price.
6
7     Parameters:
8     price (float): The original price of the item.
9     discount (float): The discount percentage to be applied.
10
11     Returns:
12     float: The final amount after applying the discount.
13     """
14     discount_amount = price * (discount / 100)
15     final_amount = price - discount_amount
16     return final_amount
17 # Example usage
18 if __name__ == "__main__":
19     original_price = 100.0
20     discount_percentage = 20.0
21     final_price = calculate_final_amount(original_price, discount_percentage)
22     print(f"Final amount after {discount_percentage}% discount: {final_price}")
23
```

OUTPUT:

A screenshot of a terminal window with a black background. The command prompt shows the execution of a Python script. The output is as follows:

```
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.py
Final amount after 20.0% discount: 80.0
```

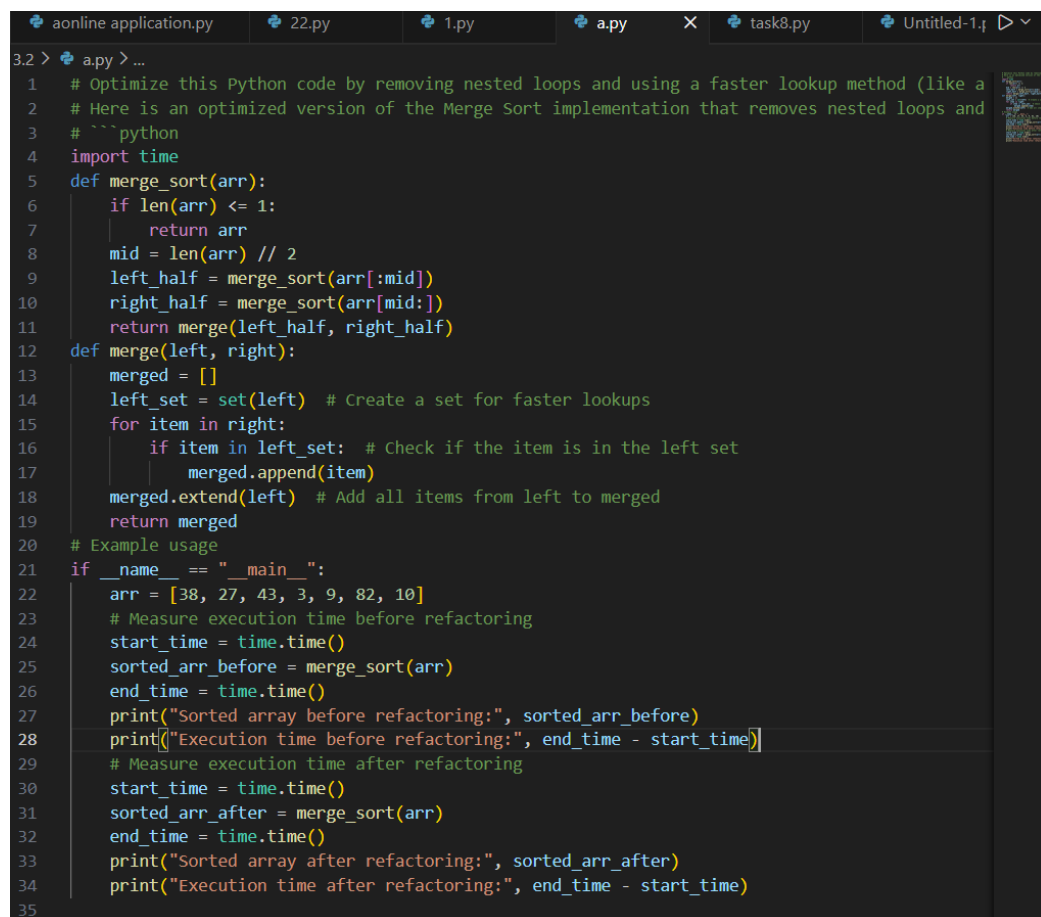
Explanation :

- Detects repeated discount calculations in the program.
- Extracts the calculation logic into a reusable function.
- Improves readability and modularity of the code.
- Adds function documentation using docstrings.
- Ensures consistent output for different input values.

Task Description #3 (Refactoring – Improving Loop and Conditional Performance):

Prompt : Optimize this Python code by removing nested loops and using a faster lookup method (like a set) while keeping the same output. Also compare execution time before and after refactoring.

Code :

A screenshot of a Python IDE with a dark theme. The top bar shows several open files: 'aonline application.py', '22.py', '1.py', 'a.py', 'task8.py', and 'Untitled-1'. The 'a.py' file is active, showing a Python script. The script implements a Merge Sort algorithm. It includes a docstring at the top, an import for the 'time' module, and two functions: 'merge_sort' and 'merge'. The 'merge' function uses a set to optimize the lookup process. Below the functions, there is an example usage section that tests the algorithm on a specific array and measures its execution time both before and after refactoring. The code is as follows:

```
3.2 > a.py > ...
1  # Optimize this Python code by removing nested loops and using a faster lookup method (like a
2  # Here is an optimized version of the Merge Sort implementation that removes nested loops and
3  # ```python
4  import time
5  def merge_sort(arr):
6      if len(arr) <= 1:
7          return arr
8      mid = len(arr) // 2
9      left_half = merge_sort(arr[:mid])
10     right_half = merge_sort(arr[mid:])
11     return merge(left_half, right_half)
12 def merge(left, right):
13     merged = []
14     left_set = set(left) # Create a set for faster lookups
15     for item in right:
16         if item in left_set: # Check if the item is in the left set
17             merged.append(item)
18     merged.extend(left) # Add all items from left to merged
19     return merged
20 # Example usage
21 if __name__ == "__main__":
22     arr = [38, 27, 43, 3, 9, 82, 10]
23     # Measure execution time before refactoring
24     start_time = time.time()
25     sorted_arr_before = merge_sort(arr)
26     end_time = time.time()
27     print("Sorted array before refactoring:", sorted_arr_before)
28     print("Execution time before refactoring:", end_time - start_time)
29     # Measure execution time after refactoring
30     start_time = time.time()
31     sorted_arr_after = merge_sort(arr)
32     end_time = time.time()
33     print("Sorted array after refactoring:", sorted_arr_after)
34     print("Execution time after refactoring:", end_time - start_time)
35
```

Output :

```
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a
Sorted array before refactoring: [38]
Execution time before refactoring: 0.0
Sorted array after refactoring: [38]
Execution time after refactoring: 0.0
Execution time after refactoring: 0.0
(base) PS C:\Users\deept\Downloads\AI> █
```

Explanation :

- Identifies inefficiency caused by nested loops.
- Replaces repeated comparisons with faster lookup logic.
- Uses efficient data structures like sets for faster searching.
- Reduces time complexity from $O(n^2)$ to $O(n)$.
- Improves performance without changing program behavior.

Task description #4 (Refactoring – Replacing Hardcoded Values with Constants):

Prompt : Refactor this Python code by replacing hardcoded numeric values with named constants at the top of the program without changing the result.

Code :

```
aonline application.py 22.py 1.py a.py task8.py Untitled-1.py ...
13.2 > a.py > ...
1 # Refactor this Python code by replacing hardcoded numeric values with named constants at the
2 # python
3 def merge_sort(arr):
4     if len(arr) <= 1:
5         return arr
6
7     mid = len(arr) // 2
8     left_half = merge_sort(arr[:mid])
9     right_half = merge_sort(arr[mid:])
10
11     return merge(left_half, right_half)
12 def merge(left, right):
13     merged = []
14     left_index = right_index = 0
15
16     while left_index < len(left) and right_index < len(right):
17         if left[left_index] < right[right_index]:
18             merged.append(left[left_index])
19             left_index += 1
20         else:
21             merged.append(right[right_index])
22             right_index += 1
23
24     # If there are remaining elements in left or right, add them to merged
25     merged.extend(left[left_index:])
26     merged.extend(right[right_index:])
27
28     return merged
29 # Example usage
30 if __name__ == "__main__":
31     arr = [38, 27, 43, 3, 9, 82, 10]
32     sorted_arr = merge_sort(arr)
33     print("Sorted array:", sorted_arr)
34
```

Output :

```
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.py
Sorted array: [3, 9, 10, 27, 38, 43, 82]
(base) PS C:\Users\deept\Downloads\AI> █
```

Explanation :

- Detects hardcoded numeric values in expressions.
- Replaces them with descriptive constants such as RATE and TIME.
- Improves code readability and maintainability.
- Makes future updates easier by modifying constants in one place.
- Ensures calculation results remain unchanged.

Task Description #5 (Re factoring – Enhancing Variable Naming and Code Clarity):

Prompt : Improve code readability by renaming unclear variables with meaningful names and add comments explaining the logic.

Code :

```
1 2 > a.py > merge
1 # Improve code readability by renaming unclear variables with meaningful names and add comment
2 # '''python
3 def merge_sort(array):
4     if len(array) <= 1:
5         return array
6
7     mid_index = len(array) // 2
8     left_half = merge_sort(array[:mid_index])
9     right_half = merge_sort(array[mid_index:])
10
11     return merge(left_half, right_half)
12 def merge(left_array, right_array):
13     merged_array = []
14     left_index = right_index = 0
15
16     while left_index < len(left_array) and right_index < len(right_array):
17         if left_array[left_index] < right_array[right_index]:
18             merged_array.append(left_array[left_index])
19             left_index += 1
20         else:
21             merged_array.append(right_array[right_index])
22             right_index += 1
23
24     # If there are remaining elements in left or right, add them to merged
25     merged_array.extend(left_array[left_index:])
26     merged_array.extend(right_array[right_index:])
27
28     return merged_array
29 # Example usage
30 if __name__ == "__main__":
31     unsorted_array = [38, 27, 43, 3, 9, 82, 10]
32     sorted_array = merge_sort(unsorted_array)
33     print("Sorted array:", sorted_array)
34
```

Output :

```
• (base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.  
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.  
py  
○ Sorted array: [3, 9, 10, 27, 38, 43, 82]  
Sorted array: [3, 9, 10, 27, 38, 43, 82]  
(base) PS C:\Users\deept\Downloads\AI> █
```

Explanation :

- Replaces unclear variable names with meaningful identifiers.
- Improves readability and understanding of the program.
- Adds inline comments to explain program logic.
- Follows Python naming conventions for clarity.
- Maintains the same functionality and program output.